



CompSci 230 Object Oriented Software Development

Object-oriented Programming Concepts & Introduction to Java



Today's Agenda

► Topics:

- Object-oriented Programming Concepts
- How does Java compare with Python?
- Anatomy of Java
- Setting Up and Getting Started in Java Programming
 - Notepad++, TextPad, Eclipse
- Console Output

► Reading:

- The Java Tutorials
 - Lesson: Object-Oriented Programming Concepts
 - <https://docs.oracle.com/javase/tutorial/java/concepts/index.html>
- Welcome to Java for Python Programmers
 - <http://interactivepython.org/courselib/static/java4python/index.html>
- Java how to program Late objects version (D & D)
 - Chapter I



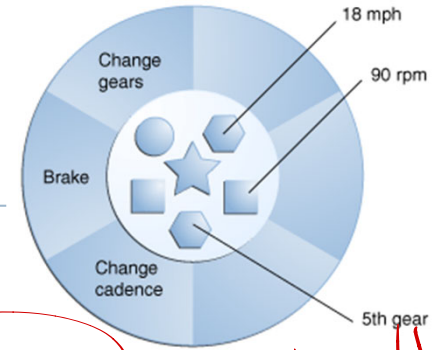
1. Object-oriented Programming Concepts

- ▶ Focuses on how these concepts relate to the real world, while simultaneously providing an introduction to the syntax of the Java programming language
- ▶ Concepts:
 - ▶ Objects,
 - ▶ Classes,
 - ▶ Inheritance, and
 - ▶ Interfaces



1. OO Programming Concepts

Objects and Classes



- ▶ What Is an Object? *class Student* : *id, name*
 - ▶ An object is a software bundle of related state and behavior. *enroll*
 - ▶ Software objects are often used to model the real-world objects that you find in everyday life.
 - ▶ explains how **state** and **behavior** are represented within an object,
 - ▶ introduces the concept of **data encapsulation**, and
 - ▶ explains the benefits of **designing** your software in this manner.
- ▶ What Is a Class?
 - ▶ A class is a blueprint or prototype from which objects are created.
 - ▶ **defines** a class that models the state and behavior of a real-world object.
 - ▶ It intentionally focuses on the basics, showing how even a simple class can cleanly model state and behavior.

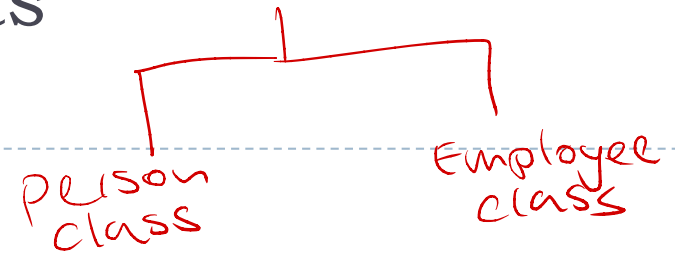


1. OO Programming Concepts

Inheritance & Interface

class of a class →

Person class



► What Is Inheritance?

- Inheritance provides a powerful and natural mechanism for **organizing** and **structuring** your software.
 - explains how classes **inherit** state and behavior from their superclasses, and
 - explains how to **derive** one class from another using the simple syntax provided by the Java programming language.

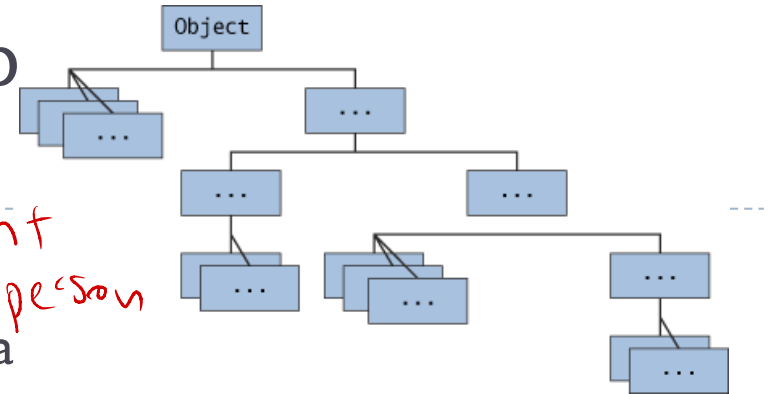
► What Is an Interface?

- An interface is a **contract** between a class and the outside world.
 - When a class implements an interface, it **promises** to provide the behavior published by that interface.
 - **defines** a simple interface and explains the **necessary changes** for any class that implements it.



1. OO Programming Concept

Basic Inheritance



- ▶ Inheritance models is-a relationship
 - ▶ For example: a sedan is a car, a sports car is a car, a student is a person
- ▶ Inheritance allows you to define a very **general** class then later define more specialized classes by adding new detail
 - ▶ The general class is called the base or parent class (superclass)
 - ▶ The specialized classes are derived from the base class. They are called derived or child classes (subclass)
- ▶ The specialized classes inherit all the properties of the general class
 - ▶ You only have to write the "difference" or "specialization" code for each derived class
- ▶ A class hierarchy: classes can be derived from derived classes (child classes can be parent classes)



2. Python Vs Java

How does Java compare with Python?

► Java:

Simple	Architecture neutral
Object oriented	Portable
Distributed	High performance(?)
Multithreaded	Robust (as <u>defined by Gosling</u>)
Dynamic	Secure

► Python:

Simpler than Java	Architecture neutral
Object oriented	Portable
Distributed	Adequate performance
Multithreaded	Less robust?
More dynamic than Java	More difficult to secure?

► Java programs are “robust” if they are well-tested: reliable behaviour.

- Python is not a strongly-typed language, so a method can produce strange results if given an unexpected input. More difficult to test, so less “robust”?



2. Python Vs Java

▶ Python:

- ▶ the syntax is sparse, and clear.
- ▶ the underlying model of how objects and variables work is very consistent.
- ▶ You can write powerful and interesting programs without a lot of work.

▶ Java:

- ▶ For very large programs Java and C++ are going to give you the best performance.
- ▶ Maintainability is very important too.



2. Python Vs Java

Dynamic vs Static Typing

int x = 0;

- ▶ It is the way that each language handles variables. *cannot x = 'hello'.*
- ▶ **Java** forces you to define the type of a variable when you first declare it and will not allow you to change the type later in the program.
 - ▶ This is known as static typing.
- ▶ **Python** uses dynamic typing, which allows you to change the type of a variable, by replacing an integer with a string. *but python*
x = 10
x = "hello" ✓
- ▶ Pros & Cons
 - ▶ Dynamic typing is easier for the novice programmer to grips with
 - ▶ without worrying too much about their types
 - ▶ However,
 - ▶ Static typing reduces the risk of **undetected** errors plaguing your program
 - ▶ It is easy to misspell a variable name and accidentally create a whole new variable



2. Python Vs Java

Braces vs Indentation

▶ Python:

- ▶ uses indentation to separate code into blocks

▶ Java:

- ▶ uses curly braces to define the beginning and end of each function and class definition

```
public class MyProgram {  
    public static void main(String[] args) {  
        System.out.println("Hello world!");  
    }  
}
```

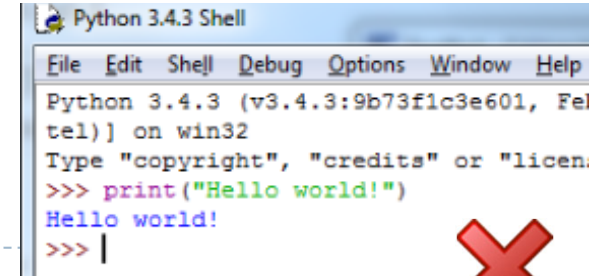
This area is the body of the class.



2. Python Vs Java

Hello World program

- ▶ Note: Python has a command-line interface which will execute a single line of code immediately after you type it



```
Python 3.4.3 Shell
File Edit Shell Debug Options Window Help
Python 3.4.3 (v3.4.3:9b73f1c3e601, Feb 2015) on win32
Type "copyright", "credits" or "license()" for more
>>> print("Hello world!")
Hello world!
>>> |
```



Java doesn't accept code from the command line!

- ▶ Python Source code: Hello.py
 - ▶ Print "Hello World!"

```
def main():
    print("Hello world!")

main()
```

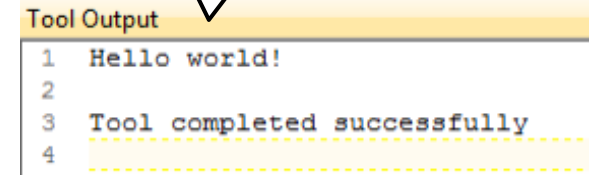
Python

- ▶ Java Source code: MyProgram.java
 - ▶ Print "Hello World!"

Java

```
public class MyProgram {
    public static void main(String[] args) {
        System.out.println("Hello world!");
    }
}
```

Output

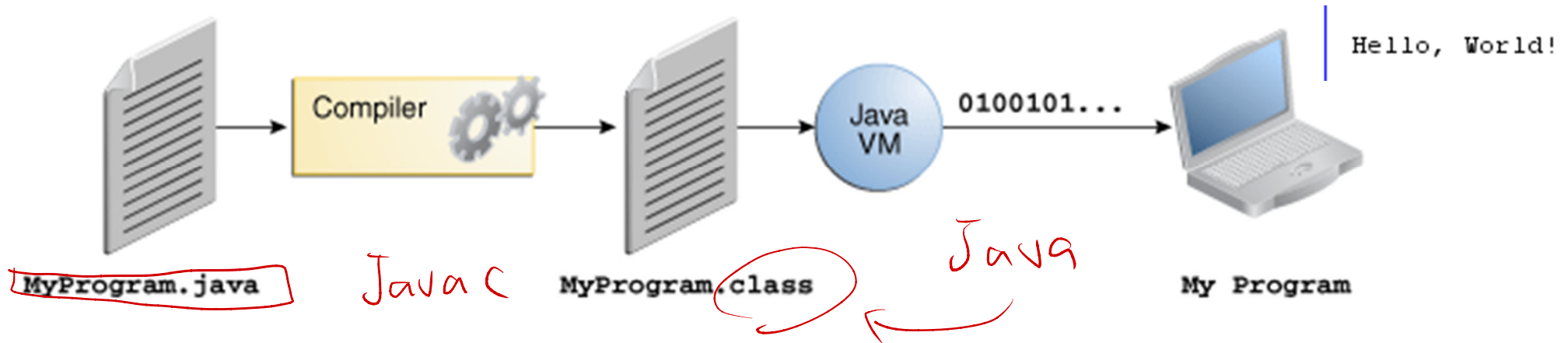


```
Tool Output
1 Hello world!
2
3 Tool completed successfully
4
```



3. Anatomy of Java

Java: A Compiled and Interpreted Language



- ▶ “In the Java programming language, all source code is first written in plain text files ending with the `.java` extension.”
 - ▶ “Those source files are then **compiled** into `.class` files by the `javac` compiler.”
- ▶ “A `.class` file does not contain code that is native to your processor;
 - ▶ “it instead contains bytecodes — the machine language of the Java Virtual Machine (Java VM).”
 - ▶ “The java launcher tool then **runs** your application [by **interpreting** its bytecode on] an instance of the Java Virtual Machine.”
- ▶ Source: <http://docs.oracle.com/javase/tutorial/getStarted/intro/definition.html>



3. Anatomy of Java

Java – a mixed system



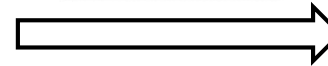
Java source code (MyProgram.java)

```
public class MyProgram{  
    public static void main(String[] args) {  
        System.out.println("Hello world!");  
    }  
}
```

Java byte code (MyProgram.class)

javac.exe

COMPILER



1001	0010	0111
1101	0110	0011
1011	0110	0111
0001	0010	0111
1011	0110	0101



java.exe

INTERPRETER
(JVM for Computer 1)



Computer 1

java.exe

INTERPRETER
(JVM for Computer 2)



Computer 2

java.exe

INTERPRETER
(JVM for Computer 3)



Computer 3



3. Anatomy of Java

Important Rules

- ▶ A Java source code file contains one or more Java classes.
 - ▶ If more than one class is in a source code file, only one of them may be public.
- ▶ The public class and the filename of the source code file must match.
 - ▶ ex: A class named MyProgram must be in a file named MyProgram.java
- ▶ Every Executable Java program must have a function called: **public static void main(String[] args)**

File name

MyProgram.java

Class name

Method name

```
public class MyProgram {  
    * public static void main(String[] args) {  
        System.out.println("Hello world!");  
    }  
}
```

This is the method header for the main method. The main method is where a Java application begins.

This area is the body of the main method. All of the actions to be completed during the main method will be between these curly braces.



3. Anatomy of Java Comments

▶ Comments:

- ▶ Ignored by the compiler
- ▶ Aimed at people who read the code

▶ Inline comments

```
//comment goes here
```

- ▶ All text until end of line is ignored e.g.

▶ Multi-line comments

- ▶ All text between start and end of comment is ignored e.g.

```
/* comment goes  
   in this space, here */
```

▶ Javadoc comments

- ▶ Delimited by `/**` and `*/`.
- ▶ Enable you to embed program documentation directly in your programs.
- ▶ The **javadoc** utility program (online Appendix G) reads Javadoc comments and uses them to prepare program documentation in HTML format.

```
/** documentation */
```



3. Anatomy of Java Javadoc & API

- ▶ Classes are grouped into packages
 - ▶ named groups of related classes
 - ▶ It includes all Java packages, classes, and interfaces, along with their methods, fields, and constructors.
 - ▶ and are collectively referred to as the Java class library, or the Java Application Programming Interface (Java API).
 - ▶ A complete listing of all classes in Java API can be found at Oracle's website:
<http://docs.oracle.com/javase/7/docs/api/>
 - ▶ You use **import** declarations to identify the predefined classes used in a Java program.
 - ▶ For example:
 - `java.util.Scanner;`
 - allows a program to accept input from the keyboard
 - `java.lang`
 - fundamental to the design of Java language. i.e. is automatically imported in a Java program and
 - does not need an explicit import statement.

```
import java.util.Scanner;
```

```
import java.util.*;
```

import
all classes
from the utility
package



3. Anatomy of Java

Parts of a Java Program

▶ Class Header

- ▶ The class header tells the compiler things about the class such as what other classes can use it (public) and that it is a Java class (class), and the name of that class (MyProgram).

▶ Curly Braces

- ▶ When associated with the class header, they define the scope of the class.
- ▶ When associated with a method, they define the scope of the method.

▶ The main Method

- ▶ This line must be exactly as shown in the example (except the args variable name can be programmer defined).
- ▶ This is the line of code that the java command will run first.
- ▶ This method starts the Java program.
- ▶ Every Java application must have a main method.



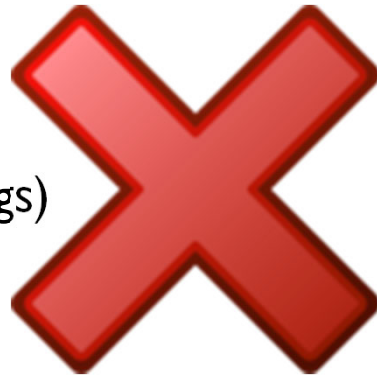
3. Anatomy of Java

Structure of a Java program

- ▶ public static void main(String[] args):

- ▶ the following lines look similar but are in fact treated by Java as completely different methods

- ▶ public void main(String[] args)
- ▶ public static void main(String args)
- ▶ public static void main()
- ▶ void main(String args)



- ▶ **public** indicates to the Java compiler that this is a method that anyone can call
- ▶ **static** tells Java that this is a method that is part of the class, but is not a method for any one instance of the class
- ▶ **void** tells the Java compiler that the method main will not return a value
- ▶ **args:** It is a parameter list for the main method. It is an array of Strings



3. Anatomy of Java

Blocks of code



- ▶ Whitespace, indentation etc. is for human readability only! It has no meaning for the compiler!

- ▶ Blocks go inside matching pairs of curly braces { and }. Blocks can be nested.

- ▶ { } { }

```
public class  MyProgram {  
    public static void main(String[] args) {  
        System.out.println("Hello world!");  
    }  
}
```

*both
fine*

```
public class  MyProgram { public static void main(String[] args) {  
    System.out.println("Hello world!");  } }
```

- ▶ Both examples are valid

- ▶ Whitespace have no syntactic significance in Java. But they help your program more human-readable.
 - ▶ i.e. we prefer to use the first version!



3. Anatomy of Java Compile and Run

How to Run Java
program

- ▶ You can use any IDE (e.g.: Notepad++, Textpad, Eclipse, etc)
 - ▶ Please check our course website to get information regarding to setting up Notepad++ and textpad for Java programming
 - ▶ You can add commands to compile and run java programs

▶ All Java files have the extension .java.

▶ Start cmd

▶ Go to the current directory:

```
cd Desktop
```

```
Administrator: Command Prompt
Microsoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.

C:\Users\kng001.UOA>java -version
java version "1.8.0_131"
Java(TM) SE Runtime Environment (build 1.8.0_131-b11)
Java HotSpot(TM) 64-Bit Server VM (build 25.131-b11, mixed mode)

C:\Users\kng001.UOA>cd Desktop
C:\Users\kng001.UOA\Desktop>javac MyProgram.java
C:\Users\kng001.UOA\Desktop>
```

▶ To compile Java programs we use the **javac** command,

```
javac MyProgram.java
```

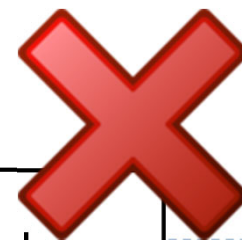
▶ To run an application we use the **java** command, e.g.

```
java MyProgram
```

Note: we only use the class
name: "MyProgram"

```
C:\Users\kng001.UOA\Desktop>java MyProgram
Hello world!
```

Never use:
java MyProgram.class
java MyProgram.java





3. Anatomy of Java Errors

► There are three kinds of errors:

► Syntax errors,

- These are errors where Java finds something wrong with your program, and you can't execute it.
- The compiler will tell you where it got into trouble. Usually the error is on the exact line indicated by the compiler, or on the line just before it;

Won't generate class file by the compiler

► Runtime errors, and

- If there are no syntax errors, Java may detect an error while your program is running
- For example: divide by 0 etc

► Logic errors.

- A logical error, or bug, is when your program compiles and runs, but does the wrong thing.
- difficult to find and correct.

Generate unexpected output



3. Anatomy of Java Syntax

- ▶ **Syntax:** The set of legal structures and commands that can be used in a particular language.
 - ▶ Every basic Java statement ends with a semicolon ;
 - ▶ The contents of a class or method occur between { and }
- ▶ **Syntax error (compiler error):** A problem in the structure of a program that causes the compiler to fail.
 - ▶ **Examples:**
 - ▶ Missing semicolon ✓
 - ▶ Too many or too few { } braces ✓
 - ▶ Illegal identifier for class name
 - ▶ Class and file names do not match
 - ▶ ...



3. Anatomy of Java

Syntax error example

```
1 public class Hello {  
2     pooblic static void main(String[] args) {  
3         System.owt.println("Hello, world!")_  
4     }  
5 }
```

► Compiler output:

```
Hello.java:2: <identifier> expected  
    pooblic static void main(String[] args) {  
      ^  
Hello.java:3: ';' expected  
    }  
    ^  
2 errors
```

- The compiler shows the line number where it found the error.
 - The error messages can be tough to understand!
-



3. Anatomy of Java

Example: Using NotePad++

- ▶ Two steps:
 - ▶ Compile the Java file
 - ▶ Run the program

The screenshot displays the Notepad++ interface with a Java file named `Test.java` open. The code defines a `Test` class with a `main` method that prints "Hello world!". The `Macro` menu is open, showing options like `Compile` (Ctrl+1), `Run` (Ctrl+2), and `Compile and Run` (Ctrl+3). Below the editor, the `Console` window shows the output of the compilation and execution process, including the command `javac Test.java` and the output `Process finished. ***** READY *****`.

```
public class Test {  
    public static void main(String[] args) {  
        System.out.println("Hello world!");  
    }  
}
```

Console Output:

```
NPP_EXEC: "Compile"  
CD: C:\Users\paul\Desktop  
Current directory: C:\Users\paul\Desktop  
javac Test.java  
Process started >>>  
<<< Process finished.  
***** READY *****
```



3. Anatomy of Java

Example: Using TextPad

- ▶ Two steps:
 - ▶ Compile the Java file
 - ▶ Run the program

```
Test.java ×
1 public class Test {
2     public static void main(String[] args) {
3         System.out.println("Hello world!");
4     }
5 }
6
```

Tools Macros Configure Window Help

- Compare Files... Ctrl+F9
- Spelling... F7
- Sort... F9
- Windows Explorer Alt+F3
- Stop Ctrl+Break
- Run...
- External Tools
 - Compile Java Ctrl+1
 - Run Java Application Ctrl+2

Tool Output

```
1
2 Tool completed successfully
3
```

Tools Macros Configure Window Help

- Compare Files... Ctrl+F9
- Spelling... F7
- Sort... F9
- Windows Explorer Alt+F3
- Stop Ctrl+Break
- Run...
- External Tools
 - Compile Java Ctrl+1
 - Run Java Application Ctrl+2

Tool Output

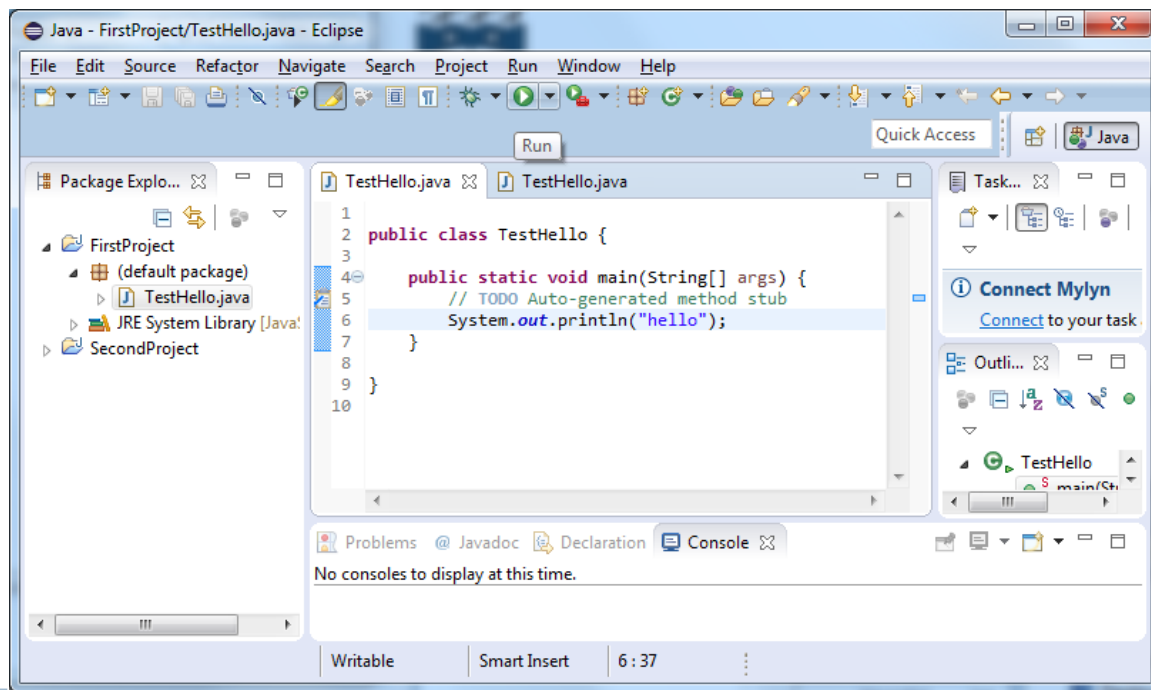
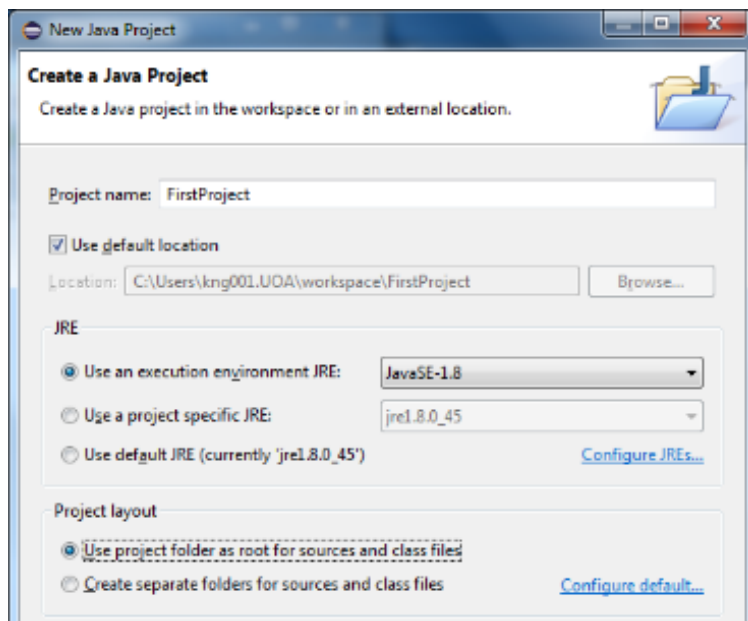
```
1 Hello world!
2
3 Tool completed successfully
4
```



3. Anatomy of Java

Example: Using Eclipse

- ▶ You will need to create a new Java project for each of your Java application
 - ▶ Create a new class/program
 - ▶ Click the “Run” button to compile and run the program

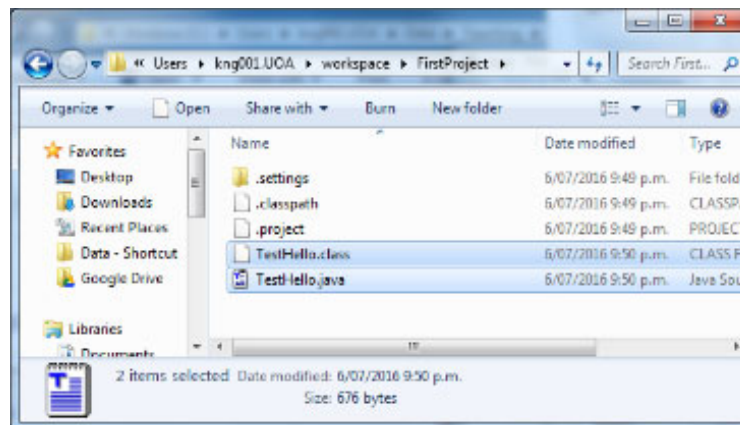
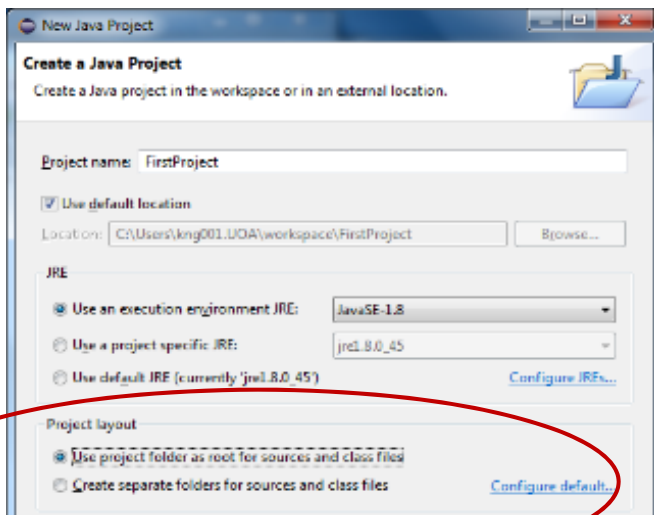




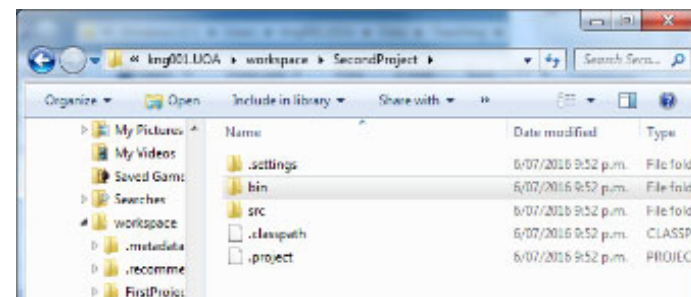
3. Anatomy of Java

Example: Using Eclipse

- ▶ **Note:** You may want to change the Project Layout setting:
 - ▶ Use **project folder as root for sources and class files**
 - ▶ All java files and class files are within the root directory



- ▶ If you choose “Create separate folders...”
 - ❑ The bin folder contains all class files
 - ❑ The src folder contains all java files

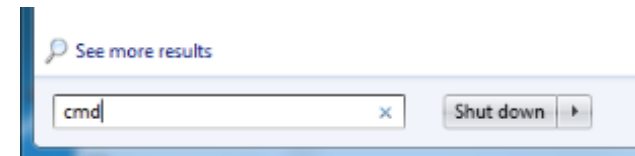




4. Setting up your computer

► Installing Java:

- <http://www.oracle.com/technetwork/java/javase/downloads/index.html>
- Choose **Java SE Downloads** -> Java Platform (JDK) 8u241 or SE 11
- Click “Accept License Agreement”
- Choose Platform and the version number (JDK 8/11)
 - There are several equivalent names
 - Java Standard Edition Development Kit / Java SE Development Kit / JDK
- Usually Java is installed in the "Program Files" folder in the C drive of your computer.
- Have you installed the JDK correctly?
 - Type “cmd” in the start menu
 - Enter “java -version” in the command prompt window. It should print the version information.



```
Select Administrator: Command Prompt
Microsoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.

C:\Users\kng001\UOA>java -version
java version "1.8.0_131"
Java(TM) SE Runtime Environment (build 1.8.0_131-b11)
Java HotSpot(TM) 64-Bit Server VM (build 25.131-b11, mixed mode)
```



4. Setting Up and Getting Started Common Problems (and Their Solutions)

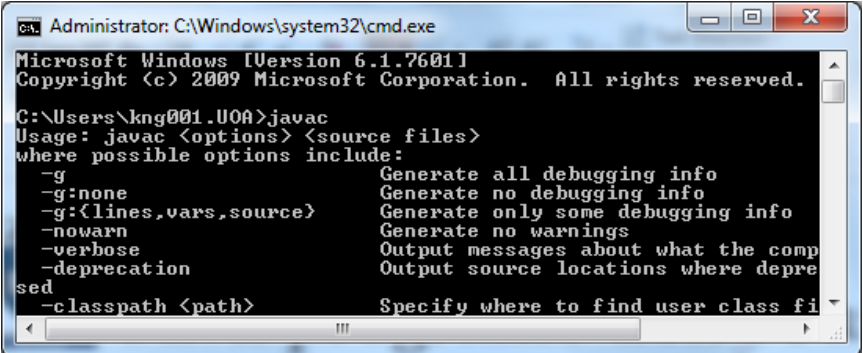
► Compiler Problems:

- Common Error Messages on Microsoft Windows Systems
- 'javac' is not recognized as an internal or external command, operable program or batch file *path is not set properly!*
- If you receive this error, Windows cannot find the compiler (javac).



4. Setting Up and Getting Started Solution:

- ▶ You may have problem in your Path variable on your system.
- ▶ Check the following
 - ▶ In the start menu, type “cmd”
 - ▶ Type “javac” + press ENTER key
 - ▶ It should display “Usage: javac...”
 - ▶ Otherwise, follow the steps below:
- ▶ How to fix it:
 - ▶ Go to the **C** drive, open the **Program Files** folder, open the **Java** folder and open the jdk version folder (jdk1.8.x_x) and open the bin folder:
 - ▶ Check that java.exe and javac.exe are in this folder
 - ▶ Now copy the path to the JDK version you are running, e.g.



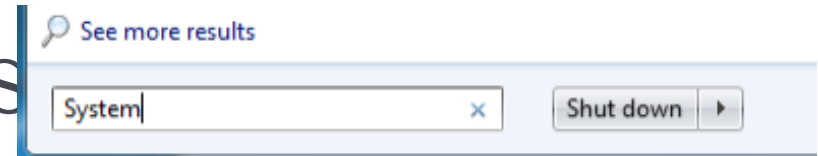
```
Administrator: C:\Windows\system32\cmd.exe
Microsoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.

C:\Users\kng001.U0A>javac
Usage: javac <options> <source files>
where possible options include:
  -g               Generate all debugging info
  -g:none          Generate no debugging info
  -g:{lines,vars,source}  Generate only some debugging info
  -nowarn          Generate no warnings
  -verbose         Output messages about what the compiler is doing
  -deprecation     Output source locations where deprecated APIs are found
  -classpath <path> Specify where to find user class files
```

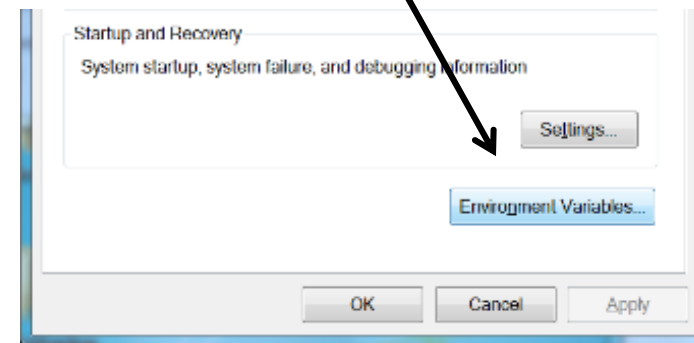
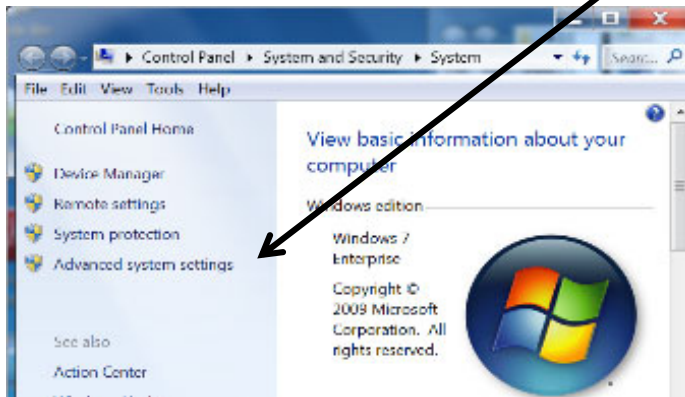
C:\Program Files\Java\jdk1.8.0_161\bin

C:\Program Files\Java\jdk-11.0.3\bin

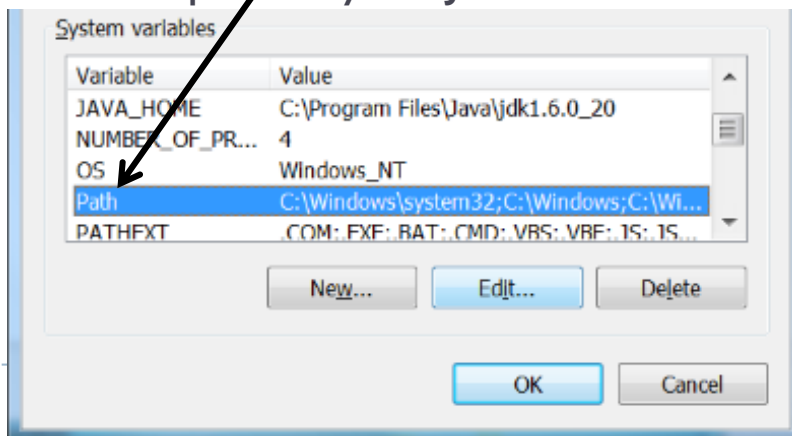
4. Setting Up and Getting Solution



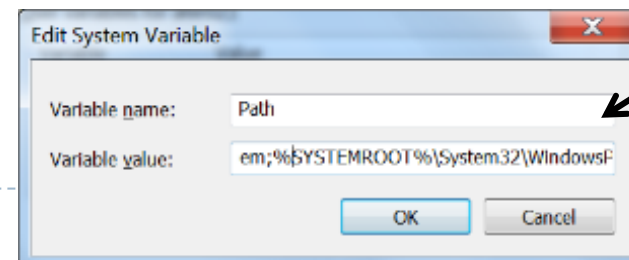
- ▶ Type "System" in the start menu and choose System from Control Panel
- ▶ Click "Advanced system settings" on the left menu
- ▶ Select the "Advanced" tab and click "Environment Variables" button



- ▶ Edit the Path System variable
 - ▶ Insert the path to your JDK after one of the first ";". Your path must be between two ";".



...;SYSTEMROOT%\System32; C:\Program Files\Java\jdk1.xxx\bin





4. Setting Up and Getting Common Problem:

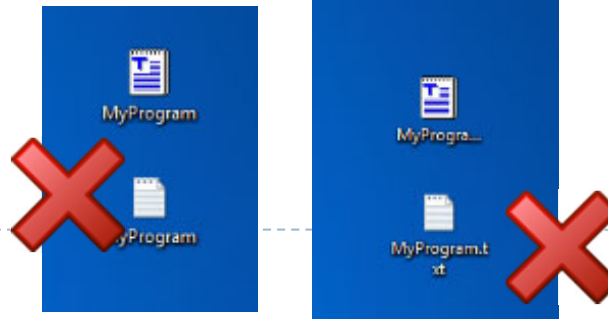
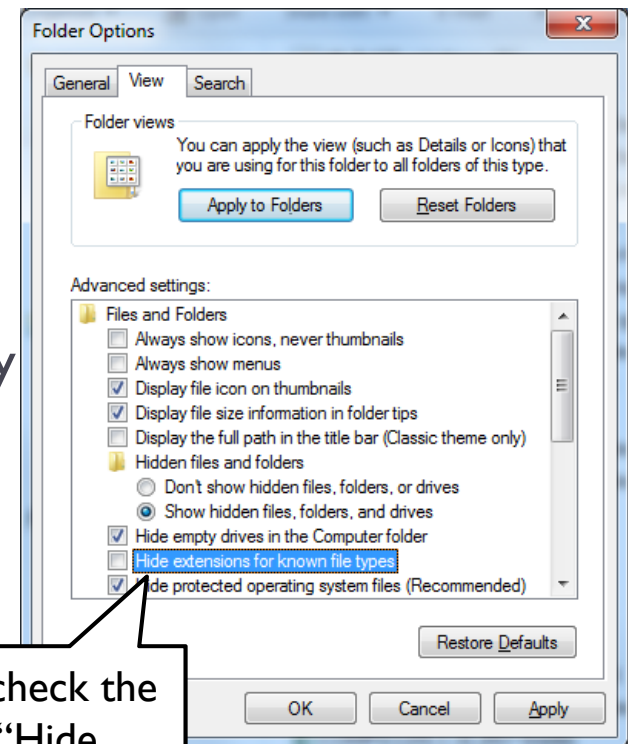
```
public class MyProgram {  
    public static void main(String[] args) {  
        System.out.println("Hello world!");  
    }  
}
```

- Compiler Problems: file not found:

```
javac MyProgram.java
```

```
C:\Users\kng001.UOA\Desktop>javac MyProgram.java  
javac: file not found: MyProgram.java  
Usage: javac <options> <source files>  
use -help for a list of possible options
```

- Check your filename carefully.
 - You may hide the extensions for all known file types by the default setting of your computer.
 - Go to your current folder, select **Organize** from the menu, choose **Folder and search options**
 - Select the “**View**” tab, uncheck the “Hide extensions for known file types”





4. Setting Up and Getting Started

Common Problem – Running a program

▶ Error message:

- ▶ Exception in thread "main" java.lang.NoClassDefFoundError: HelloWorldApp/class
- ▶ If you try to run your program with `java HelloWorldApp.class` instead of `java HelloWorldApp`.
- ▶ Remember, the argument is the name of the class that you want to use, not the filename.

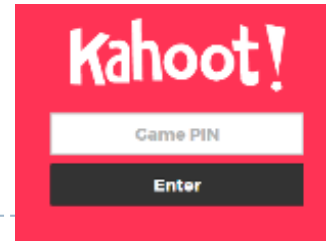
▶ Error message:

- ▶ Exception in thread "main" java.lang.NoSuchMethodError: main
- ▶ The Java VM requires that the class you execute with it have a **main** method at which to begin execution of your application.

```
public class MyProgram {  
    public static void main(String[] args) {  
        System.out.println("Hello world!");  
    }  
}
```

main method

Exercise 1



- ▶ What is missing in the following Java program? Can you compile and run it?

```
public class MyProgram1 {  
    public static void main(String args) {  
        System.out.println("Hello world!");  
    }  
}
```

[] away



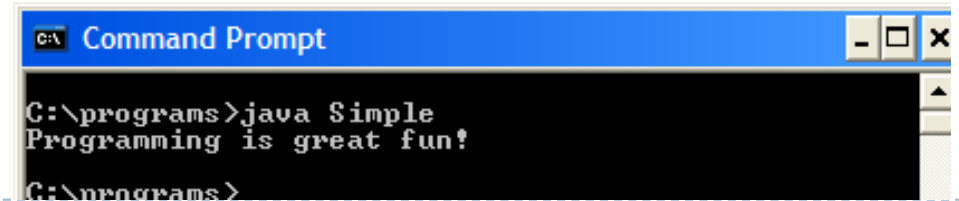
- ▶ Can you compile and run the following Java program successfully?

```
public class MyProgram2 {  
    public static void Main(String[] args) {  
        System.out.println("Hello world!");  
    }  
}
```

main



5. Console Output Printing to Console



```
C:\> Command Prompt
C:\programs>java Simple
Programming is great fun!
C:\programs>
```

- ▶ Java sends information to the standard output device by using a Java *class* stored in the standard Java library
- ▶ Java classes in the standard Java library are accessed using the Java Applications Programming Interface (API).
- ▶ Example:

Simple Java

```
public class Simple {
    public static void main(String[] args) {
        System.out.println("Programming is great fun!");
    }
}
```

- ▶ This line uses the **System** class from the standard Java library.
- ▶ The **System** class contains methods and objects that perform system level tasks.
- ▶ The **out** object, a member of the **System** class, contains the methods **print** and **println**.



5. Console Output println VS print

- ▶ The print and println methods actually perform the task of sending characters to the output device.
- ▶ The println method places a newline character at the end of whatever is being printed out.

```
System.out.println("This is being printed out");  
System.out.println("on two separate lines.");
```

- ▶ The print statement works very similarly to the println statement.
 - ▶ However, the print statement does not put a newline character at the end of the output.

```
System.out.print("These lines will be");  
System.out.print("printed on");  
System.out.println("the same line.");
```

These lines will beprinted onthe same line.



5. Console Output

Java Escape Sequences

// is a string not ''

- ▶ There are some special characters that can be put into the output.
- ▶ they are treated by the compiler as a single character.

<code>\n</code>	newline	Advances the cursor to the next line for subsequent printing
<code>\t</code>	tab	Causes the cursor to skip over to the next tab stop
<code>\b</code>	backspace	Causes the cursor to back up, or move left, one position
<code>\r</code>	carriage return	Causes the cursor to go to the beginning of the current line, not the next line
<code>\\</code>	backslash	Causes a backslash to be printed
<code>\'</code>	single quote	Causes a single quotation mark to be printed
<code>\"</code>	double quote	Causes a double quotation mark to be printed

```
System.out.println("Hello, \"World\"!");
```

Hello, "World"!



Exercise 2



Modifying Your First Java Program

► 1) Displaying a Single Line of Text with Multiple Statements

- Display “Welcome to Java Programming” (one line of text) using two statements:

```
public class MyProgram3 {  
    public static void main(String[] args) {  
        //complete this  
  
    }  
}
```

Welcome to Java Programming

► 2) Displaying Multiple Lines of Text with a Single Statement

```
public class MyProgram4 {  
    public static void main(String[] args) {  
        //complete this  
  
    }  
}
```

Welcome
to
Java
Programming!



Summary

- ▶ Java is a case-sensitive language.
- ▶ All Java programs must be stored in a file with a .java file extension.
- ▶ A .java file may contain many classes but may only have one public class.
- ▶ If a .java file has a public class, the class must have the same name as the file.
- ▶ Java applications must have a main method.
- ▶ For every left brace, or opening brace, there must be a corresponding right brace, or closing brace.
- ▶ Statements are terminated with semicolons.
 - ▶ Comments, class headers, method headers, and braces are not considered Java statements.



Things to DO

- ▶ Download and Install JDK into your own laptop if needed
- ▶ Read Chapter 1 from D & D
- ▶ Read The Java Tutorials
 - ▶ Lesson: Object-Oriented Programming Concepts
- ▶ Complete CodeRunner exercises